Sistema de Gestión de Inventarios basado en Arquitectura de Microservicios

Entrega 1: Investigación y fundamentación

Caso de estudio: Distribuidora El Montañero S.A.S.

Luis Alejandro Saraza Marín, Brayan David Collazos Escobedo y Mateo Falla Muñoz

Departamento de Informática y Computación, Facultad de Administración

Universidad Nacional de Colombia, Sede Manizales

Ingeniería de Software II

José Albeiro Montes Gil

24 de octubre de 2026

Tabla de contenido

Investigación de sistemas similares 4
InvenTree 4
Part-DB 6
Zoho Inventory 7
Cuadro comparativo 9
Conclusiones de la investigación 10
Definición del problema y alcance 11
Empresa caso de estudio 11
Situación actual 11
Planteamiento del problema 12
Objetivo del sistema 12
Actores y roles 12
Alcance del sistema 13
Fuera del alcance 13
Justificación técnica de la arquitectura 14
Microservicios frente a monolito 14
Decisión 15
Identificación preliminar de microservicios 16
Dominios del negocio 16
Gestión de usuarios en el API Gateway 17
Vista preliminar de la arquitectura 18
Mecanismo de comunicación 18
Pila tecnológica 19
Criterio de asignación 19
Asignación por componente 19
Flujo de autenticación 20
Requerimientos 21
Requerimientos funcionales 21
Requerimientos no funcionales 24
Repositorio y organización del equipo 27
Bitácora de uso de IA 29
Reflexión final de la fase 31
Referencias 33

Sistema de Gestión de Inventarios basado en Arquitectura de Microservicios

Este documento corresponde a la primera entrega del proyecto de aula del curso Ingeniería de Software II, desarrollado bajo la metodología de Aprendizaje Basado en Proyectos (ABP). En la fase de investigación y fundamentación, el equipo analiza tres sistemas de gestión de inventarios existentes, define el problema y el alcance del sistema para la empresa caso de estudio, justifica técnicamente la elección de una arquitectura de microservicios frente a una monolítica, identifica de forma preliminar los microservicios y las tecnologías con las que se construirán, y especifica los requerimientos funcionales y no funcionales. El documento cierra con la bitácora de uso de inteligencia artificial (IA) de la fase.

El código fuente, la documentación y las bitácoras del proyecto se alojan en el repositorio del equipo: https://github.com/Mat-FM/Sistemas_Gestor_Inventario. La Tabla 1 relaciona cada entregable exigido por el enunciado con la sección del documento que lo desarrolla.

Tabla 1

Entregables de la Entrega 1 y su ubicación en el documento

Entregable exigido por el enunciado	**Sección del documento**
Investigación de al menos tres sistemas similares, con comparativo de funcionalidades, fortalezas y debilidades	Investigación de sistemas similares
Definición del problema y alcance del sistema	Definición del problema y alcance
Justificación técnica de microservicios frente a monolito	Justificación técnica de la arquitectura
Identificación preliminar de los microservicios	Identificación preliminar de microservicios; Pila tecnológica
Documento de requerimientos funcionales y no funcionales	Requerimientos
Repositorio de GitHub con actividad de todos los integrantes	Repositorio y organización del equipo
Bitácora de uso de IA de la fase	Bitácora de uso de IA

Investigación de sistemas similares

Se analizaron tres sistemas representativos de distintos modelos de distribución: dos de código abierto e instalados en servidores propios (InvenTree y Part-DB) y uno comercial ofrecido como servicio en la nube (Zoho Inventory). Los datos de versiones, precios y funcionalidades se tomaron de la documentación oficial consultada en septiembre de 2026, y las fuentes se verificaron nuevamente el 24 de octubre de 2026. Las versiones indicadas corresponden a las vigentes en septiembre de 2026.

InvenTree

InvenTree es un sistema de gestión de inventarios de código abierto que ofrece control detallado de existencias y seguimiento de partes (InvenTree, s. f.-e). Está construido en Python con el framework Django, ofrece una interfaz web y una API REST, y puede ampliarse mediante complementos (*plugins*) (InvenTree, s. f.-e). Sus desarrolladores lo presentan como una herramienta intuitiva, pensada tanto para empresas como para aficionados (InvenTree, s. f.-d).

Tabla 2

Ficha técnica de InvenTree

Aspecto	Detalle
Tipo y licencia	Código abierto (licencia MIT), instalado en servidor propio (InvenTree, s. f.-e)
Versión estable	1.5.6, publicada el 26 de septiembre de 2026 (InvenTree, 2026)
Tecnología	Python, Django y Django REST Framework en el servidor; React en la interfaz (InvenTree, s. f.-e)
Base de datos	PostgreSQL, MySQL o SQLite (InvenTree, s. f.-e)
API	REST con autenticación por token (InvenTree, s. f.-b)
Despliegue	Docker Compose o instalador para Linux (InvenTree, s. f.-a, s. f.-c)
Costo	Sin costo de licencia

Funcionalidades principales

Organiza los productos, a los que llama «partes», en categorías, con atributos, precios y unidades de medida (InvenTree, s. f.-c, s. f.-i). Controla las existencias por ubicación con ajustes, órdenes

de transferencia, trazabilidad, historial de movimientos y control de vencimiento (InvenTree, s. f.-c). En compras gestiona proveedores y órdenes de compra con recepción de mercancía (InvenTree, s. f.-j); en ventas, clientes, órdenes de venta y órdenes de devolución (InvenTree, s. f.-k). Incluye además manufactura con listas de materiales y órdenes de fabricación, reportes y etiquetas basados en plantillas, lectura de códigos de barras y una aplicación móvil (InvenTree, s. f.-c).

Cada parte puede tener un stock mínimo: cuando la existencia total cae por debajo de ese valor, la parte se marca como de stock bajo en la interfaz (InvenTree, s. f.-i), y los usuarios suscritos a la parte o a su categoría reciben una notificación por correo electrónico (InvenTree, s. f.-h). Una tarea diaria en segundo plano avisa también sobre existencias próximas a vencer (InvenTree, s. f.-l).

Fortalezas

- Licencia MIT y API REST documentada como base para integraciones (InvenTree, s. f.-b, s. f.-e).
- Procesamiento en segundo plano: un proceso dedicado ejecuta las tareas que no conviene atender en el servidor web, como el envío de correos y la generación de reportes (InvenTree, s. f.-f).
- Alertas por stock mínimo y por vencimiento (InvenTree, s. f.-h, s. f.-l).
- Desarrollo activo: en 2026 publicó las versiones 1.2, 1.3, 1.4, 1.5 y 1.6 (InvenTree, s. f.-g).

Debilidades

- Operación compleja: la instalación con Docker levanta cuatro contenedores obligatorios (base de datos, servidor web, proceso en segundo plano y servidor intermedio) y uno opcional de memoria caché (InvenTree, s. f.-a).
- Orientado a manufactura: buena parte de sus módulos (listas de materiales, órdenes de fabricación) no aplica a una distribuidora.

Part-DB

Part-DB es un sistema de inventarios de código abierto orientado a componentes electrónicos, que se instala en un servidor web y se usa desde el navegador. Lo emplean aficionados y espacios de fabricación colaborativa (*makerspaces*), pero también pequeñas empresas y universidades (Part-DB, s. f.-e). Su concepto central es la «parte», un tipo de artículo cuyas existencias se distribuyen en lotes, cada uno con una ubicación y una cantidad (Part-DB, s. f.-b).

Tabla 3

Ficha técnica de Part-DB

Aspecto	Detalle
Tipo y licencia	Código abierto (licencia AGPL-3.0), instalado en servidor propio (Part-DB, s. f.-e)
Versión estable	2.19.0, publicada el 27 de septiembre de 2026 (Part-DB, 2026)
Tecnología	PHP 8.2 o superior con el framework Symfony; Bootstrap 5 y Hotwire en la interfaz (Part-DB, s. f.-e)
Base de datos	MySQL, MariaDB, PostgreSQL o SQLite (Part-DB, s. f.-e)
API	REST con tokens de permisos limitados y servidor OAuth 2.0 (Part-DB, s. f.-a, s. f.-d)
Despliegue	Servidor web, imagen Docker o Kubernetes mediante Helm (Part-DB, s. f.-c, s. f.-e)
Costo	Sin costo de licencia

Funcionalidades principales

Cada parte se asocia a una categoría, un tipo de encapsulado (*footprint*), un fabricante, varias ubicaciones e información de precios, y admite adjuntos como hojas de datos o imágenes. El sistema maneja usuarios y grupos con permisos detallados, autenticación de doble factor e inicio de sesión único (SSO) mediante SAML. Su registro de eventos guarda qué cambios hace cada usuario y permite devolver las partes a versiones anteriores. Ofrece búsqueda por especificaciones técnicas, generación de etiquetas con códigos de barras, lectura de códigos con la cámara web, importación y exportación de datos, e integración con el programa de diseño electrónico KiCad y con proveedores en línea como Digikey o LCSC (Part-DB, s. f.-e). Sin embargo, el campo

de stock mínimo figura en la documentación como no implementado por completo (Part-DB, s. f.-b).

Fortalezas

- Seguridad: permisos detallados por grupo y autenticación de doble factor (Part-DB, s. f.-e).
- Auditoría: registro de eventos con posibilidad de revertir cambios (Part-DB, s. f.-e).
- Despliegue sencillo, con soporte oficial para Kubernetes (Part-DB, s. f.-c).
- Interfaz adaptable, utilizable desde computador, tableta o celular (Part-DB, s. f.-e).

Debilidades

- Dominio especializado en componentes electrónicos (Part-DB, s. f.-e).
- Su documentación no describe órdenes de compra, devoluciones ni reportes de inventario, y el stock mínimo está incompleto (Part-DB, s. f.-b, s. f.-e).
- El español no figura entre los idiomas soportados (Part-DB, s. f.-e).

Zoho Inventory

Zoho Inventory es un software comercial de gestión de inventarios que Zoho Corporation ofrece como servicio en la nube y que hace parte de su familia de aplicaciones Zoho Finance, junto con Zoho Books, Zoho Billing y Zoho Commerce, entre otras (Zoho Corporation, s. f.-i).

Tabla 4

Ficha técnica de Zoho Inventory

Aspecto	**Detalle**
Tipo	Software comercial de pago, ofrecido como servicio en la nube (SaaS)
Costo (edición EE. UU.)	Plan gratuito (50 órdenes, 1 usuario y 1 bodega) y planes pagos entre USD 29 y USD 249 mensuales por organización, con facturación anual (Zoho Corporation, s. f.-c)
Usuarios por plan	1 (gratuito), 3 (Standard), 5 (Premium) y 10 (Plus y Enterprise) (Zoho Corporation, s. f.-c)
Plataformas	Web y aplicaciones para iOS y Android (Lavanyasree S., 2018; Zoho Corporation, 2026)

Aspecto	Detalle
API	REST con OAuth 2.0 (Zoho Corporation, s. f.-d); límite de 100 peticiones por minuto por organización (Zoho Corporation, s. f.-b)
Infraestructura	Administrada por Zoho en ocho regiones (Zoho Corporation, s. f.-b)

Funcionalidades principales

Gestiona artículos, grupos y artículos compuestos, niveles de reorden, seguimiento por lote y por número de serie, ajustes, conteos de stock y códigos de barras, con varias bodegas y órdenes de transferencia (Zoho Corporation, s. f.-c). Al activar la notificación de reorden, el sistema envía un correo cuando un artículo cae por debajo de su nivel de reorden (Zoho Corporation, s. f.-a); además, el módulo de reabastecimiento lista los artículos con stock bajo y facilita crear la orden de compra o de transferencia correspondiente (Zoho Corporation, s. f.-e). Sus reportes incluyen resumen y antigüedad del inventario, valorización por FIFO o por costo promedio ponderado y clasificación ABC (Zoho Corporation, s. f.-f, s. f.-g). El acceso se controla con roles predefinidos y personalizados (Zoho Corporation, s. f.-i).

Fortalezas

- Es el más completo de los tres sistemas en compras, reabastecimiento y reportes (Zoho Corporation, s. f.-e, s. f.-f).
- No exige infraestructura propia y cuenta con aplicaciones móviles nativas (Lavanyasree S., 2018; Zoho Corporation, 2026).
- API documentada, con OAuth 2.0 y límites de uso explícitos (Zoho Corporation, s. f.-b, s. f.-d).
- Sitio oficial disponible en español (Zoho Corporation, s. f.-h).

Debilidades

- Pago recurrente y límites de órdenes, usuarios y bodegas según el plan (Zoho Corporation, s. f.-c).
- Código cerrado y dependencia directa del proveedor.
- Ninguna de sus ocho regiones está en Latinoamérica: Estados Unidos, Europa, India,

Australia, Japón, Canadá, China y Arabia Saudita (Zoho Corporation, s. f.-b).

Cuadro comparativo

La Tabla 5 consolida las tres dimensiones exigidas por el enunciado: funcionalidades, arquitectura y experiencia de usuario.

Tabla 5

Comparativo de los sistemas analizados

Criterio	**InvenTree**	**Part-DB**	**Zoho Inventory**
Funcionalidades			
Catálogo y stock	Partes, categorías, existencias por ubicación	Partes y lotes por ubicación	Artículos, lotes, series y varias bodegas
Órdenes de compra	Sí, con recepción	No documentadas	Sí, con reabastecimiento
Devoluciones	Órdenes de devolución	No documentadas	Sí
Reportes	Plantillas configurables	No documentados	Valorización FIFO y promedio, ABC, antigüedad
Alertas de stock	Correo a usuarios suscritos	Stock mínimo incompleto	Correo al llegar al nivel de reorden
Vencimientos	Aviso diario de stock próximo a vencer	No documentado	Seguimiento por lotes
Usuarios y permisos	Permisos por usuario	Grupos con permisos detallados y doble factor	Roles predefinidos y personalizados
Arquitectura			
Licencia	MIT	AGPL-3.0	Propietaria, por suscripción
Infraestructura	Servidor propio	Servidor propio	Nube de Zoho
Estilo	Monolito con proceso en segundo plano	Monolito (Symfony)	No publicado
Despliegue	Docker Compose con cuatro contenedores y uno opcional	Imagen Docker o Kubernetes con Helm	Administrado por el proveedor
API	REST con tokens	REST con tokens y OAuth 2.0	REST con OAuth 2.0 y límite de peticiones
Base de datos	Una sola para toda la aplicación	Una sola para toda la aplicación	Administrada por el proveedor

Criterio	InvenTree	Part-DB	Zoho Inventory
Experiencia de usuario			
Idioma español	Traducción comunitaria	No soportado	Sitio oficial en español
Plataformas	Web y aplicación móvil complementaria	Web adaptable	Web y aplicaciones iOS y Android
Códigos de barras	Sí	Sí, con cámara web	Sí
Puesta en marcha	Compleja (varios servicios)	Media (un contenedor)	Inmediata (registro en línea)

Nota. Elaboración propia con base en las fuentes citadas en las Tablas 2, 3 y 4 y en el texto de cada sistema.

Conclusiones de la investigación

1. **Existe un núcleo funcional común:** productos, existencias por ubicación, proveedores, usuarios con permisos, búsqueda y una API. Es la línea base del alcance del sistema propio.
2. **Compras, devoluciones y reportes distinguen una solución completa de una básica.** Zoho Inventory es el referente en estas áreas; de él se toman la recepción contra orden de compra, el reabastecimiento a partir del nivel de reorden y la valorización del inventario.
3. **Hay un espacio poco cubierto en la distribución de productos de consumo.** Las opciones de código abierto están pensadas para manufactura y electrónica (InvenTree, s. f.-e; Part-DB, s. f.-e), y ninguna de las tres propone la salida del lote más próximo a vencer (en inglés, FEFO), que es esencial para una distribuidora de productos empacados.
4. **Usar contenedores no equivale a tener microservicios.** Part-DB es un monolito y aun así se despliega en contenedores y en Kubernetes (Part-DB, s. f.-c). Por eso la justificación de la arquitectura propia se apoya en el escalado y el despliegue independientes y en el aislamiento de fallos. De InvenTree, que separa en un proceso aparte las tareas en segundo plano (InvenTree, s. f.-f), se toma la idea de llevar a ser-

vicios propios las tareas que no deben frenar la operación principal: notificaciones y reportes.

5. **La seguridad se concentra en el punto de entrada de la API.** Los tres sistemas protegen su API con tokens u OAuth 2.0, y Zoho limita además las peticiones por organización y responde con el código HTTP 429 al superar el límite (Zoho Corporation, s. f.-b). Ese esquema orienta el diseño del API Gateway propio (RNF09 y RNF19).

Definición del problema y alcance

Empresa caso de estudio

Distribuidora El Montañero S.A.S. es una empresa ficticia ubicada en Manizales, Caldas, que distribuye insumos y productos empacados a tiendas de barrio y cafeterías del Eje Cafetero. Opera con dos bodegas, la principal en Manizales y una satélite en Chinchiná; maneja alrededor de 800 referencias de unas 40 marcas y atiende cerca de 150 clientes. Su equipo está conformado por seis personas: un administrador, dos auxiliares de bodega, dos vendedores y un contador externo.

Se eligió esta empresa porque su tamaño hace visible el problema sin volverlo inabarcable: tiene más de una ubicación física, alta rotación de mercancía, productos con fecha de vencimiento y un equipo pequeño que no puede dedicar a nadie de tiempo completo a cuadrar el inventario.

Situación actual

El control de inventario se lleva en un archivo de Excel compartido por correo, acompañado de un cuaderno de entradas y salidas en cada bodega. De este esquema se derivan seis problemas:

1. **No hay una única versión de la información.** Cada bodega actualiza su propia copia del archivo y la consolidación se hace una vez por semana; durante seis días nadie conoce el saldo real.
2. **Los vendedores comprometen mercancía inexistente.** Consultan una hoja desactualizada y prometen productos ya vendidos, lo que genera pedidos incompletos y devoluciones.

3. **Las compras se deciden por intuición.** Sin un histórico confiable de consumo, se sobre abastecen referencias de baja rotación mientras se agotan las de alta.
4. **No hay trazabilidad.** Cuando falta mercancía no se sabe quién la movió ni cuándo; los faltantes aparecen en el conteo anual y se asumen como pérdida.
5. **Los vencimientos se detectan tarde.** Los productos próximos a vencer se identifican al revisar físicamente el estante.
6. **Los traslados entre bodegas quedan sin registro.** La mercancía que sale de Manizales no figura en ningún saldo hasta que alguien la registra en Chinchiná, a veces días después.

Planteamiento del problema

El Montañero no dispone de información de inventario confiable, oportuna ni trazable. Esto produce ventas comprometidas sin respaldo real, compras mal dimensionadas, pérdidas por vencimiento y faltantes sin responsable identificable. El control manual actual no escala con el crecimiento en número de referencias, bodegas y usuarios que trabajan de forma simultánea.

Objetivo del sistema

Construir un sistema web de gestión de inventarios, basado en una arquitectura de microservicios, que centralice el saldo de existencias por producto y bodega en tiempo real, registre cada movimiento con su responsable, anticipe los quiebres de stock y los vencimientos, y entregue indicadores que sustenten las decisiones de compra.

Actores y roles

Tabla 6

Actores del sistema y permisos generales

Rol	Quién lo ocupa	Permisos principales
Administrador	Administrador de la empresa	Acceso total: usuarios (incluido el restablecimiento de contraseñas), catálogo, proveedores, órdenes de compra, umbrales de stock, reportes y configuración.

Rol	Quién lo ocupa	Permisos principales
Operador de bodega	Auxiliares de bodega	Registrar entradas, salidas, ajustes, traslados, devoluciones, recepciones y conteos físicos en su bodega.
Vendedor	Vendedores	Consultar el catálogo y las existencias por bodega en tiempo real.
Contador	Contador externo	Consultar el kardex, el panel de auditoría y la valorización; exportar reportes.

Alcance del sistema

El sistema incluye:

- Catálogo de productos, categorías y proveedores.
- Control de existencias por producto y bodega: entradas, salidas, ajustes y traslados con estado «en tránsito».
- Gestión de varias bodegas y consulta de existencias por ubicación.
- Órdenes de compra a proveedores con recepción de mercancía total o parcial.
- Usuarios con los roles de la Tabla 6, inicio de sesión y permisos, gestionados por el API Gateway.
- Alertas por stock bajo el umbral mínimo y por productos próximos a vencer.
- Kardex completo: quién, cuándo, qué producto y qué tipo de movimiento.
- Reportes de existencias, valorización y rotación, con panel de indicadores.
- Conteo físico programado con generación automática de los ajustes.
- Carga inicial de productos y saldos desde archivos Excel o CSV.
- Devoluciones de clientes y hacia proveedores, reflejadas en las existencias.
- Interfaz adaptable, utilizable desde tableta o celular dentro de la bodega.

Fuera del alcance

Para que el proyecto sea realizable en un semestre, quedan excluidos:

- Facturación electrónica, integración con la DIAN, cartera de clientes y contabilidad general.

- Gestión de pedidos de venta y despachos; las salidas se registran como movimientos de inventario con un documento de referencia.
- Aplicaciones móviles nativas; el acceso móvil se resuelve con la interfaz web adaptable.
- Integración con tiendas en línea, plataformas de comercio electrónico o pasarelas de pago.
- Lectores de código de barras dedicados y predicción de demanda con modelos estadísticos.
- Recuperación de contraseña por correo electrónico; el restablecimiento lo hace el administrador (RF08).

Justificación técnica de la arquitectura

Microservicios frente a monolito

El sistema comprende dominios de negocio diferenciados (catálogo, inventario, proveedores, compras, notificaciones y reportes, además de la gestión de usuarios, que se resuelve en el API Gateway) con formas de uso distintas: la consulta de existencias es frecuente y la hacen varias personas al mismo tiempo, mientras que los reportes son esporádicos, pero exigen más procesamiento. La Tabla 7 compara ambas arquitecturas con los criterios técnicos del enunciado, aplicados al caso de El Montañero, e incluye los criterios en los que el monolito resulta superior.

Tabla 7

Comparación de arquitecturas para el caso de estudio

Criterio	Arquitectura monolítica	Arquitectura de microservicios
Escalabilidad	Hay que aumentar la capacidad de toda la aplicación, aunque solo la consulta de existencias tenga mucha demanda.	Se aumenta solo el servicio con más demanda, por ejemplo ms-inventario durante la jornada de ventas.
Mantenibilidad	Un cambio en un módulo puede afectar a otros por dependencias internas poco visibles.	Cada servicio tiene un dominio acotado y una API definida; modificar el catálogo no altera el kardex.

Criterio	Arquitectura monolítica	Arquitectura de microservicios
Despliegue independiente	Actualizar una función exige volver a desplegar todo el sistema y detener la operación en ambas bodegas.	Se actualiza un servicio sin interrumpir los demás, por ejemplo, cambiar los reportes durante un conteo físico.
Tolerancia a fallos	Un error grave, como un reporte que agota la memoria, puede detener el sistema completo y bloquear el registro de mercancía.	Los fallos quedan aislados: si fallan notificaciones o reportes, se siguen registrando movimientos (RNF18).
Trabajo en equipo	Los tres integrantes trabajan sobre el mismo código, con más conflictos al integrar los cambios.	Cada integrante es responsable de servicios concretos (Tabla 8) y trabaja en paralelo sobre API acordadas.
Base de datos	Una sola base compartida; las consultas de los reportes compiten con el registro de movimientos.	Cada servicio tiene su propia base de datos (RNF02); los cálculos pesados de los reportes se hacen en ms-reportes y no en la base de inventario.
Complejidad de operación	Ventaja del monolito: una sola aplicación que desplegar y vigilar.	Más componentes: red, contenedores, archivos de configuración y registros repartidos. Se mitiga con Kubernetes y con Docker Compose en desarrollo.
Consistencia de datos	Ventaja del monolito: una operación que toca varios módulos se guarda completa o no se guarda.	Una operación que involucra dos servicios puede quedar a medias si uno falla. Se mitiga confirmando la recepción de mercancía solo cuando inventario responde y con claves únicas que evitan duplicados al reintentar (RNF17).
Tiempo de respuesta	Ventaja del monolito: los módulos se comunican dentro del mismo programa.	Cada llamada entre servicios pasa por la red y suma tiempo. Se mitiga con tiempos máximos de espera, pocas llamadas por operación y el criterio de 2 s (RNF06).

Decisión

Con seis usuarios actuales, un monolito bien organizado en módulos sería técnicamente viable. Aun así, se elige la arquitectura de microservicios por cuatro razones concretas. Primera, los dominios tienen niveles de demanda muy distintos, lo que hace valioso escalar por separado la consulta de existencias. Segunda, el aislamiento de fallos protege la operación crítica, el registro de movimientos, frente a componentes no críticos como notificaciones y reportes. Tercera, la empresa proyecta crecer en bodegas y usuarios, y cada servicio puede evolucionar sin volver a de-

splegar el sistema completo. Cuarta, permite que los tres integrantes desarrollen en paralelo. Los costos reconocidos en la Tabla 7 se asumen de forma consciente y se mitigan con los requerimientos RNF10, RNF17 y RNF18.

Identificación preliminar de microservicios

Dominios del negocio

El sistema se divide en dominios del negocio (en inglés, *bounded contexts*), cada uno a cargo de un servicio. Cada microservicio, y también el API Gateway, es dueño exclusivo de sus datos, y ningún otro componente accede a su base de datos (RNF02). La Tabla 8 asigna cada requerimiento a un componente responsable.

Tabla 8

Componentes, responsabilidades y requerimientos asignados

Componente	Responsabilidad	Datos que posee	Requerimientos	Responsable
api-gateway	Único punto de entrada: dirige cada petición al servicio correspondiente; gestiona usuarios y roles, inicio y cierre de sesión, y cambio y restablecimiento de contraseña; verifica el token y los permisos; limita la cantidad de peticiones y asigna un identificador a cada una.	Usuario, Rol	RF07, RF08, RNF07, RNF09, RNF19	Por asignar
ms-catalogo	Productos, categorías y subcategorías; búsqueda y filtrado.	Producto, Categoría	RF01, RF02, RF12	Por asignar
ms-proveedores	Proveedores y su relación con los productos.	Proveedor, ProveedorProducto	RF03	Por asignar
ms-inventario	Bodegas, saldos, movimientos, traslados, lotes con salida del más próximo a vencer (FEFO), kardex, devoluciones de clientes y conteo físico. Detecta cuando un saldo queda por debajo del umbral.	Bodega, Saldo, Movimiento, Lote, Traslado, Conteo	RF04, RF05, RF09, RF10, RF13, RF15, RF17, RF18	Por asignar

Componente	Responsabilidad	Datos que posee	Requerimientos	Responsable
ms-compras	Órdenes de compra, cambios de estado, recepción total o parcial y devoluciones a proveedor.	OrdenCompra, DetalleOrden, Recepción	RF06, RF13	Por asignar
ms-notificaciones	Entrega de alertas en la interfaz y por correo; revisión diaria de vencimientos y de productos bajo el umbral.	Notificación, Preferencia	RF09, RF17	Por asignar
ms-reportes	Reportes, panel de indicadores, exportación a CSV y PDF, e importación masiva validada.	Importación	RF11, RF14, RF15, RF16	Por asignar

Nota. La asignación de responsables por componente se definirá en una entrega posterior.

Gestión de usuarios en el API Gateway

La gestión de usuarios no tiene un microservicio propio: está a cargo del API Gateway, por tres razones. Primera, el enunciado permite que el Gateway centralice aspectos comunes a todo el sistema, como la autenticación. Segunda, el Gateway ya debe revisar en cada petición quién la hace y qué permisos tiene; si él mismo administra los usuarios, no necesita consultar a otro servicio para iniciar sesión. Tercera, se reduce en uno el número de servicios que hay que construir, desplegar y probar.

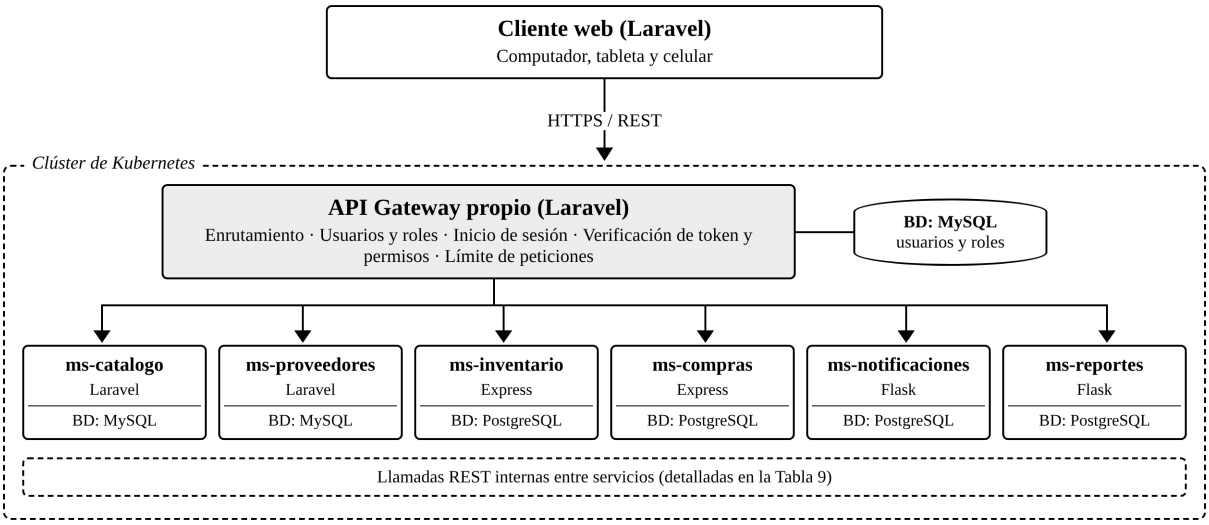
El Gateway tendrá su propia base de datos MySQL con los usuarios y sus roles, y ningún otro servicio accederá a ella (RNF02). Los microservicios no guardan usuarios: reciben en cada petición el identificador y el rol de quien la hace, enviados por el Gateway. La contraseña olvidada no se recupera por correo electrónico; el administrador la restablece con una contraseña temporal, que el usuario debe cambiar en su siguiente inicio de sesión (RF08). Como costo, el Gateway asume más funciones y se vuelve un componente crítico; por eso es uno de los dos componentes que escalan automáticamente (RNF05) y su código se organiza en capas separadas para enrutamiento, seguridad y gestión de usuarios (RNF13).

Vista preliminar de la arquitectura

La Figura 1 presenta los componentes y sus relaciones. El diagrama detallado, con contratos de API y modelos de datos, corresponde a la Entrega 2.

Figura 1

Vista preliminar de componentes



Nota. Las flechas representan peticiones REST. Todas las peticiones externas entran por el API Gateway; las llamadas entre servicios se detallan en la Tabla 9. BD = base de datos.

Mecanismo de comunicación

Toda la comunicación del sistema usa peticiones REST con datos en formato JSON. Las peticiones externas siempre pasan por el Gateway, y entre servicios solo se hacen las llamadas de la Tabla 9. Cada servicio encuentra a los demás por su nombre dentro de Kubernetes (por ejemplo, http://ms-inventario:3000), sin necesidad de una herramienta adicional de descubrimiento de servicios. Se distinguen dos tipos de llamadas:

- **Llamadas que esperan respuesta:** el servicio necesita el resultado para continuar; por ejemplo, ms-inventario confirma en ms-catalogo que el producto existe antes de registrar un movimiento.
- **Avisos que no detienen la operación:** el servicio primero guarda su operación y de-

spués avisa a otro servicio. Si el destino no responde, la operación ya quedó guardada y el fallo se anota en el registro del servicio (RNF10 y RNF18). Así, un fallo en notificaciones o reportes no impide registrar movimientos.

Pila tecnológica

Criterio de asignación

De acuerdo con RNF15, los componentes se construyen exclusivamente con los tres frameworks trabajados en clase, y las bases de datos usan los dos motores vistos en el curso, MySQL y PostgreSQL. La asignación aprovecha la fortaleza de cada framework:

- **Laravel (PHP)** para el API Gateway, los servicios de registro y consulta de datos con reglas de negocio (catálogo y proveedores) y el cliente web, porque facilita la validación de los datos y el manejo de la base de datos con poco código repetido. En el Gateway, además, sus filtros de peticiones (middleware) permiten encadenar la verificación del token, los permisos por rol y el límite de peticiones, e incluye el cifrado de contraseñas con bcrypt.
- **Express (Node.js)** para los servicios que atienden muchas peticiones al mismo tiempo: la consulta de existencias, que es el servicio a escalar (RNF05), y compras, que coordina la recepción con inventario.
- **Flask (Python)** para procesamiento de datos y tareas programadas, porque Python cuenta con bibliotecas para cálculos con datos, lectura de Excel y CSV, y generación de PDF.

Asignación por componente

Tabla 9

Pila tecnológica por componente

Componente	Framework	Base de datos	Justificación
api-gateway	Laravel	MySQL	Sus filtros de peticiones (middleware) encadenan las revisiones de token, permisos y límite de peticiones; incluye cifrado de contraseñas con bcrypt y guarda usuarios y roles. Su capacidad ante muchas peticiones se cubre aumentando su número de copias (RNF05).
ms-catalogo	Laravel	MySQL	Registro y consulta de productos con validaciones, paginación y filtros combinables (RF12).
ms-proveedores	Laravel	MySQL	Registro de proveedores y su relación de muchos a muchos con los productos.
ms-inventario	Express	PostgreSQL	Servicio más consultado; sus operaciones bloquean el saldo mientras se actualiza, para evitar saldos negativos cuando dos movimientos llegan al mismo tiempo.
ms-compras	Express	PostgreSQL	Flujo de estados de la orden y coordinación de la recepción con inventario.
ms-notificaciones	Flask	PostgreSQL	Envío de correo y revisión diaria de vencimientos y de productos bajo el umbral.
ms-reportes	Flask	PostgreSQL	Cálculos de rotación y valorización; lectura y validación de Excel y CSV; exportación a PDF.
Cliente web	Laravel	Ninguna	Interfaz adaptable que se comunica exclusivamente con el Gateway.

El enrutamiento, la verificación de tokens, la gestión de usuarios y el límite de peticiones del Gateway los programa el equipo en Laravel. No se usan soluciones de gateway ya construidas por terceros, como Kong (RNF09). Las bibliotecas específicas de cada servicio se definirán en la Entrega 2.

Flujo de autenticación

1. El usuario inicia sesión en el cliente web; el Gateway verifica la contraseña contra su base de datos de usuarios y entrega un token de sesión (JWT) válido por 60 minutos, que indica quién es el usuario y qué rol tiene.

2. En cada petición siguiente, el Gateway revisa que el token sea válido y que el rol tenga permiso para esa acción; si no es así, rechaza la petición.
3. Si todo está en orden, el Gateway reenvía la petición al microservicio que corresponde e indica quién la hace. Los microservicios no son accesibles desde fuera del clúster, así que solo reciben peticiones que ya pasaron por el Gateway.

Requerimientos

Requerimientos funcionales

Los requerimientos RF01 a RF14 corresponden al alcance mínimo del enunciado y se transcriben tal como aparecen en él; RF15 a RF18 son ampliaciones del equipo derivadas del caso de estudio. Para cada requerimiento se indica el componente responsable y la prioridad, clasificada como alta (A), media (M) o baja (B).

Tabla 10

Requerimientos funcionales

Código	Requerim-iento	Descripción	Componente	Prior.
RF01	Gestión de productos	El sistema debe permitir crear, consultar, actualizar y eliminar (dar de baja) productos, incluyendo atributos como código, nombre, descripción, categoría, unidad de medida, precio y proveedor asociado.	ms-catalogo	A
RF02	Gestión de categorías	El sistema debe permitir crear, consultar, actualizar y eliminar categorías y subcategorías de productos.	ms-catalogo	A
RF03	Gestión de proveedores	El sistema debe permitir registrar, consultar, actualizar y eliminar proveedores, y asociarlos a uno o varios productos.	ms-proveedores	A
RF04	Control de stock	El sistema debe permitir registrar movimientos de inventario (entradas, salidas, ajustes y traslados) y mantener actualizado el saldo disponible de cada producto en cada bodega/almacén.	ms-inventario	A

Código	Requerimiento	Descripción	Componente	Prior.
RF05	Gestión de almacenes o bodegas	El sistema debe permitir administrar múltiples ubicaciones físicas de almacenamiento y consultar el stock de un producto por ubicación.	ms-inventario	A
RF06	Órdenes de compra	El sistema debe permitir generar, consultar y actualizar el estado de órdenes de compra a proveedores, y recibir mercancía contra dichas órdenes.	ms-compras	A
RF07	Gestión de usuarios y roles	El sistema debe permitir el registro, consulta, actualización y asignación de usuarios, con al menos dos roles (por ejemplo, administrador y operador de bodega) con permisos diferenciados.	api-gateway	A
RF08	Autenticación de usuarios	El sistema debe permitir inicio de sesión, cierre de sesión, recuperación de contraseña y cambio de contraseña.	api-gateway	A
RF09	Alertas de stock	El sistema debe notificar (en interfaz, correo o similar) cuando el nivel de stock de un producto esté por debajo de un umbral mínimo configurable.	ms-inventario, ms-notificaciones	A
RF10	Trazabilidad y auditoría	El sistema debe registrar quién, cuándo y qué movimiento realizó sobre el inventario (kardex/histórico de movimientos).	ms-inventario	A
RF11	Reportes	El sistema debe generar reportes básicos de inventario (existencias actuales, valorización del inventario, productos con mayor y menor rotación).	ms-reportes	A
RF12	Búsqueda y filtrado	El sistema debe permitir buscar y filtrar productos por nombre, código, categoría o proveedor.	ms-catalogo	A
RF13	Gestión de devoluciones	El sistema debe permitir registrar devoluciones de productos, tanto de clientes como hacia proveedores, y reflejarlas en el stock.	ms-inventario, ms-compras	A
RF14	Panel de indicadores (dashboard)	El sistema debe presentar indicadores clave del inventario (rotación, quiebres de stock, valor total, entre otros).	ms-reportes	A
RF15	Panel de auditoría	Consultar el kardex filtrando por usuario, fecha, bodega, producto y tipo de movimiento, con exportación del resultado.	ms-inventario, ms-reportes	M

Código	Requerimiento	Descripción	Componente	Prior.
RF16	Carga y descarga masiva	Importar productos, proveedores y stock inicial desde CSV o Excel con validación previa y reporte de errores por fila; las filas válidas se envían al servicio dueño de cada dato. Exportar reportes a CSV y PDF.	ms-reportes	M
RF17	Lotes, vencimiento y FEFO	Registrar lote y fecha de vencimiento, sugerir la salida del lote más próximo a vencer y alertar con una anticipación configurable en días.	ms-inventario, ms-notificaciones	A
RF18	Conteo físico	Programar conteos por bodega, registrar las cantidades contadas y generar los ajustes por diferencia con aprobación del administrador.	ms-inventario	B

Para el caso de El Montañero, el equipo precisa el alcance de algunos de los requerimientos del enunciado:

- **RF04:** los traslados entre bodegas se registran con estado «en tránsito» hasta que la bodega de destino los recibe.
- **RF06:** la recepción de mercancía contra una orden de compra puede ser total o parcial.
- **RF07:** los roles del sistema son los de la Tabla 6 (administrador, operador de bodega, vendedor y contador), y la gestión de usuarios y roles la realiza el API Gateway.
- **RF08:** la recuperación de contraseña no usa correo electrónico: el administrador restablece la contraseña con una contraseña temporal, que el usuario debe cambiar en su siguiente inicio de sesión.
- **RF09:** las alertas se muestran en la interfaz y se envían por correo, y el umbral mínimo lo configura el administrador para cada producto.
- **RF10:** además de quién, cuándo y qué movimiento, el kardex registra el producto y la bodega.
- **RF11:** la valorización se calcula como la cantidad en existencia por el precio registrado del producto, y la rotación según las salidas registradas en el periodo consul-

tado.

- **RF14:** el panel incluye también los productos próximos a vencer (RF17).

Requerimientos no funcionales

Cada requerimiento se acompaña de un criterio verificable, para que pueda comprobarse en las pruebas.

Tabla 11

Requerimientos no funcionales y criterios de verificación

Código	Requerimiento	Descripción	Criterio de verificación
RNF01	Arquitectura	El sistema debe estar construido bajo una arquitectura de microservicios, con servicios independientes y débilmente acoplados, comunicados mediante API REST y/o mensajería asíncrona.	Seis microservicios y un API Gateway propio, independientes entre sí, que se comunican mediante peticiones REST.
RNF02	Persistencia independiente	Cada microservicio debe gestionar su propia base de datos, evitando el acceso directo de un servicio a la base de datos de otro.	Cada servicio, incluido el Gateway, tiene su propia base de datos y sus propias credenciales; ningún servicio conoce las credenciales de otro ni lee su base de datos.
RNF03	Contenerización	Todos los microservicios y sus dependencias (bases de datos, colas de mensajería, etc.) deben ejecutarse en contenedores Docker.	Un Dockerfile por servicio; las bases de datos también se ejecutan en contenedores.
RNF04	Orquestación	El sistema debe desplegarse y gestionarse obligatoriamente mediante Kubernetes; el uso de Kubernetes no es opcional.	Todo el sistema se despliega en Kubernetes a partir de los archivos de configuración guardados en el repositorio.
RNF05	Escalabilidad	La arquitectura debe permitir escalar de forma independiente los microservicios con mayor carga (por ejemplo, el de consulta de stock).	ms-inventario y el Gateway pasan automáticamente de una a cuatro copias cuando el uso del procesador supera el 70 %; los servicios no guardan datos de sesión en memoria, para que cualquier copia pueda atender cualquier petición.

Código	Requerimiento	Descripción	Criterio de verificación
RNF06	Rendimiento	El sistema debe responder a las operaciones de consulta en un tiempo menor a 2 segundos bajo una carga concurrente definida y sustentada por el equipo.	Las consultas responden en menos de 2 s en el 95 % de los casos (percentil 95) y con menos de 1 % de errores, con 50 usuarios simulados al mismo tiempo durante 5 minutos. La cifra supera ocho veces la planta actual de seis personas para cubrir crecimiento y picos.
RNF07	Seguridad	Las comunicaciones y la autenticación deben protegerse mediante mecanismos estándar (por ejemplo, JWT/OAuth2, cifrado de contraseñas).	Token de sesión (JWT) firmado, con expiración de 60 minutos; contraseñas guardadas con cifrado bcrypt; la contraseña temporal de un restablecimiento debe cambiarse en el siguiente inicio de sesión.
RNF08	Documentación de API	Cada microservicio debe exponer documentación de su API (OpenAPI/Swagger o equivalente).	Especificación OpenAPI (Swagger) de cada servicio, disponible en la ruta /docs.
RNF09	API Gateway propio	Todas las peticiones dirigidas al sistema deben pasar obligatoriamente por un API Gateway construido por el equipo; no se permite el uso de Kong ni de ninguna otra solución de gateway de terceros.	El Gateway es el único componente accesible desde fuera del clúster; los microservicios solo aceptan peticiones desde dentro. No se usan Kong ni otros gateways de terceros.
RNF10	Observabilidad	El sistema debe contar con registro (logging) centralizado o consultable de cada microservicio, y de forma deseable métricas básicas de monitoreo.	Cada servicio escribe registros (logs) con el nombre del servicio, el nivel del mensaje y un identificador de petición que permite seguirla entre servicios; los registros se consultan desde Kubernetes. Cada servicio ofrece la ruta /health para comprobar que está activo.
RNF11	Pruebas de rendimiento	Cada integrante del equipo debe definir y ejecutar un mínimo de quince (15) pruebas de rendimiento sobre los endpoints que él mismo defina, con sus resultados documentados.	Quince pruebas por integrante (45 en total) en escenarios de carga normal, carga pico y estrés, con métricas y resultados documentados.
RNF12	Control de versiones en GitHub	Todo el proyecto debe alojarse en un repositorio de GitHub por equipo, en el cual debe evidenciarse la actividad (commits, ramas y pull requests) de todos los integrantes.	Commits, ramas y pull requests de los tres integrantes visibles en el repositorio de GitHub.

Código	Requerimiento	Descripción	Criterio de verificación
RNF13	Mantenibilidad	El código debe seguir buenas prácticas de diseño (bajo acoplamiento, alta cohesión).	El código de cada servicio se organiza en capas (rutas, controladores, lógica de negocio y acceso a datos) y se revisa con herramientas de análisis de código en la integración continua.
RNF14	Portabilidad	Gracias a la contenerización, el sistema debe poder desplegarse en cualquier entorno compatible con Docker y Kubernetes sin cambios en el código fuente.	Toda la configuración que cambia según el entorno (direcciones, puertos y credenciales) se entrega mediante variables de entorno y no queda escrita en el código fuente.
RNF15	Tecnologías permitidas	Únicamente pueden emplearse los lenguajes, frameworks, bases de datos y demás herramientas vistas y trabajadas durante las clases del curso.	Laravel, Flask y Express como frameworks; MySQL y PostgreSQL como bases de datos.
RNF16	Uso documentado de IA	El proceso de desarrollo debe evidenciar, mediante las bitácoras correspondientes, el uso crítico y comprendido de herramientas de inteligencia artificial.	Bitácora completa en cada una de las cuatro entregas.
RNF17	Idempotencia	Ampliación del equipo (no incluida en el enunciado).	Las peticiones que registran movimientos llevan una clave única (cabecera Idempotency-Key); si la misma petición llega dos veces, el sistema devuelve la respuesta original y no duplica el movimiento.
RNF18	Resiliencia	Ampliación del equipo (no incluida en el enunciado).	Las llamadas entre servicios esperan como máximo 3 s. Solo las operaciones que no se duplican al repetirse se reintentan, hasta tres veces, con esperas crecientes de 0,2, 0,4 y 0,8 s. Si un servicio falla cinco veces seguidas, se deja de llamar durante 30 s antes de volver a probar (cortacircuitos). El registro de movimientos sigue operando aunque notificaciones o reportes no estén disponibles.

Código	Requerimiento	Descripción	Criterio de verificación
RNF19	Límite de peticiones	Ampliación del equipo (no incluida en el enunciado).	100 peticiones por minuto por usuario autenticado y 10 intentos de inicio de sesión por minuto desde una misma dirección IP; al superar el límite, el Gateway responde con el código HTTP 429 e indica cuándo se puede volver a intentar (cabecera Retry-After).
RNF20	Pruebas en integración continua	Ampliación del equipo (no incluida en el enunciado).	Pruebas unitarias ejecutadas automáticamente en cada pull request, con cobertura mínima del 60 % en api-gateway, ms-inventario y ms-catalogo.
RNF21	Uso en dispositivos móviles	Ampliación del equipo (no incluida en el enunciado).	Interfaz utilizable en pantallas desde 360 píxeles de ancho, para su uso con celular o tableta en la bodega.

Repositorio y organización del equipo

El proyecto se aloja en el repositorio https://github.com/Mat-FM/Sistemas_Gestor_Inventario, en el cual se registra la actividad de cada integrante mediante commits, ramas y pull requests. El repositorio reúne todos los componentes en un solo lugar (monorepositorio), con la estructura de la Figura 2.

Figura 2

Estructura del repositorio

```
Sistemas_Gestor_Inventario/
|-- api-gateway/           Laravel (incluye usuarios y roles)
|-- services/
|   |-- ms-catalogo/       Laravel
|   |-- ms-proveedores/    Laravel
|   |-- ms-inventario/     Express
|   |-- ms-compras/        Express
|   |-- ms-notificaciones/ Flask
|   `-- ms-reportes/       Flask
|-- web-client/            Laravel
|-- k8s/                   Archivos de configuración de Kubernetes
|-- performance-tests/     Pruebas k6 por integrante
|-- docs/                  Documentos de cada entrega
`-- bitacoras-ia/          Bitácoras de uso de IA
```

El trabajo en el repositorio sigue estas convenciones:

- Ramas main (versión estable), develop (integración) y feature/<servicio>-<descripción> para cada funcionalidad.
- Todo cambio se integra mediante pull request hacia develop, revisado por otro integrante.
- Los mensajes de commit siguen la convención Conventional Commits (feat:, fix:, docs:).
- Cada integrante mantiene sus pruebas de rendimiento en performance-tests/<nombre>/.

Bitácora de uso de IA

Campo	Valor
Proyecto	Sistema de Gestión de Inventarios – Distribuidora El Montañero S.A.S.
Equipo / Integrantes	Luis Alejandro Saraza Marín, Brayan David Collazos Escobedo, Mateo Falla Muñoz
Entrega N.°	1
Periodo cubierto	24/08/2026 – 01/10/2026

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt utilizado (resumen)	Intervención humana	Resultado y aprendizaje
25/09/2026	Mateo Falla Muñoz	Claude	Selección y análisis de sistemas de inventario existentes	Buscar tres sistemas de gestión de inventario del mercado, preferiblemente de código abierto; describir sus funcionalidades, fortalezas y debilidades; elaborar un cuadro comparativo de funcionalidades, arquitectura y experiencia de usuario, y redactar el análisis y las conclusiones del estado del arte.	Medio	Se conservaron las características de funcionamiento e implementación de los tres sistemas y se filtró la información demasiado técnica o poco relevante. Se aprendió a especificar mejor los requerimientos en los prompts y la importancia de verificar los resultados de la IA.
25/09/2026	Luis Alejandro Saraza Marín	ChatGPT	Comparación entre arquitectura monolítica y de microservicios	A partir del enunciado del proyecto, desarrollar la justificación técnica de los microservicios frente al monolito para el sistema, comparando escalabilidad, mantenibilidad, acoplamiento, despliegue independiente, trabajo en equipo y persistencia, de forma breve, con un cuadro comparativo y sin agregar información no respaldada por el documento.	Medio	Se tomó el cuadro comparativo con ajustes y se resumieron manualmente los párrafos. Se aprendió sobre los beneficios de los microservicios para un sistema con muchas funcionalidades y su relación con Docker y Kubernetes.

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt utilizado (resumen)	Intervención humana	Resultado y aprendizaje
25/09/2026	Brayan David Collazos Escobedo	Claude	Requerimientos funcionales y no funcionales adicionales	Proponer requerimientos funcionales y no funcionales adicionales a los del enunciado que sean alcanzables en un semestre, sin ser demasiado ambiciosos.	Medio	Se analizó la lista propuesta, se seleccionaron algunos requerimientos y se ajustó su alcance para hacerlos viables en el proyecto (RF15 a RF17 y RNF17 a RNF20).
27/09/2026	Brayan David Collazos Escobedo	Gemini	Definición del problema y del alcance	A partir de los requerimientos del proyecto, crear una empresa ficticia ubicada en Manizales, con un problema por resolver y el alcance del sistema.	Bajo	El caso propuesto se ajustaba al proyecto; se modificaron los objetivos del alcance que no tenían relación con los requerimientos o eran demasiado ambiciosos para un proyecto académico.
27/09/2026	Brayan David Collazos Escobedo	Claude	Revisión de cumplimiento de la entrega, verificación de fuentes, definición de la pila tecnológica y edición del documento en LaTeX	Analizar el documento de la Entrega 1 frente al enunciado, verificar que todos los entregables estén cubiertos, proponer mejoras, especificar la pila tecnológica usando solo Laravel, Flask y Express, y generar el documento en LaTeX.	Medio	La revisión identificó referencias sin lista bibliográfica, requerimientos sin servicio responsable, requerimientos no funcionales sin métricas y una mención a facturación ajena al alcance. Al contrastar las fuentes se actualizaron las versiones de InvenTree y Part-DB y se retiraron afirmaciones no respaldadas. El equipo definió las tecnologías vistas en clase (MySQL y PostgreSQL, sin broker de mensajería), lo que llevó a adoptar el patrón Outbox para los eventos.

Fecha	Integrante	Herramienta	Tarea apoyada	Prompt utilizado (resumen)	Intervención humana	Resultado y aprendizaje
24/10/2026	Mateo Falla Muñoz	Claude	Refinamiento del documento de la Entrega 1: coherencia con el enunciado, formato APA 7, verificación de fuentes, simplificación del lenguaje, glosario de términos y retiro del microservicio de usuarios; cambio del API Gateway a Laravel, corrección de la tabla de contenido y conversión del documento a LaTeX	Analizar el enunciado y la versión preliminar; verificar la coherencia con la entrega, el cumplimiento de APA 7 en citas y formato, y la validez de las fuentes; eliminar ambigüedades pensando en las próximas entregas; reducir los términos técnicos y explicar los necesarios en un glosario aparte; eliminar ms-usuarios y trasladar la gestión de usuarios al Gateway; consultar al equipo antes de cada decisión importante, sin rehacer el documento ni sobredimensionar el proyecto. Después: cambiar el API Gateway de Express a Laravel en todo el documento, corregir la tabla de contenido y pasar la última versión a LaTeX para facilitar su visualización.	Medio	La IA en su labor de apoyo presento diversos aspectos que se salían de los requerimientos o de la idea principal del proyecto. Fue necesario guiarla y asegurarse de que no se saliera de los parámetros requeridos. Por decisión del equipo, el API Gateway pasó de Express a Laravel, y la IA ajustó la pila tecnológica, la estructura del repositorio y la Figura 1; además, corrigió la tabla de contenido, que mostraba números de página desactualizados y secciones eliminadas, y pasó el documento a LaTeX para que fuera más fácil de visualizar

Reflexión final de la fase

En esta fase delegamos a la IA tareas de exploración y redacción inicial: la búsqueda de sistemas de referencia, los borradores de la justificación arquitectónica y del caso de estudio, la propuesta de requerimientos adicionales y, al final, una revisión de cumplimiento del documento. Lo hicimos porque una primera versión rápida nos permitía discutir sobre algo concreto y dedicar el tiempo a decidir. Aprendimos que la calidad de la respuesta depende de lo específico del prompt: cuando no fijamos límites, las herramientas propusieron alcances excesivos y contenido ajeno al caso, como una referencia a facturación que no hace parte del sistema. También detectamos limitaciones concretas: versiones de software que ya habían cambiado al momento de la entrega, citas sin referencia completa y argu-

mentos genéricos que no se ajustaban a El Montañero. Para verificar, contrastamos cada dato de los sistemas analizados con su documentación oficial, revisamos cada requerimiento sugerido frente al enunciado y a nuestra capacidad real en un semestre, y ajustamos las propuestas tecnológicas a las herramientas efectivamente vistas en clase antes de incorporarlas al documento.

Referencias

InvenTree. (s. f.-a). *Docker installation guide*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/start/docker/

InvenTree. (s. f.-b). *InvenTree API*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/api/

InvenTree. (s. f.-c). *InvenTree documentation*. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/

InvenTree. (s. f.-d). *InvenTree: Intuitive inventory management*. Recuperado el 24 de octubre de 2026, de https://inventree.org/

InvenTree. (s. f.-e). *InvenTree: Open source inventory management system* [Software]. GitHub. Recuperado el 24 de octubre de 2026, de https://github.com/inventree/InvenTree

InvenTree. (s. f.-f). *InvenTree processes*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/start/processes/

InvenTree. (s. f.-g). *News*. Recuperado el 24 de octubre de 2026, de https://inventree.org/news

InvenTree. (s. f.-h). *Part notifications*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/part/notification/

InvenTree. (s. f.-i). *Parts*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/stable/part/

InvenTree. (s. f.-j). *Purchase order*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/purchasing/purchase_order/

InvenTree. (s. f.-k). *Sales*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/sales/

InvenTree. (s. f.-l). *Stock expiry*. InvenTree Documentation. Recuperado el 24 de octubre de 2026, de https://docs.inventree.org/en/latest/stock/expiry/

InvenTree. (2026, 26 de septiembre). *Release 1.5.6* [Software]. GitHub. https://github.com/inventree/InvenTree/releases/tag/1.5.6

Lavanyasree S. (2018, 21 de noviembre). *Zoho Inventory comes to your pocket with an all-new Android app*. Zoho Inventory Blog. https://www.zoho.com/inventory/blog/zoho-inventory-comes-to-your-pocket-with-an-all-new-android-app.html

Part-DB. (s. f.-a). *Authentication*. Part-DB Documentation. Recuperado el 24 de octubre de 2026, de https://docs.part-db.de/api/authentication.html

Part-DB. (s. f.-b). *Concepts*. Part-DB Documentation. Recuperado el 24 de octubre de 2026, de https://docs.part-db.de/concepts.html

Part-DB. (s. f.-c). *Kubernetes / Helm charts*. Part-DB Documentation. Recuperado el 24 de octubre de 2026, de https://docs.part-db.de/installation/kubernetes.html

Part-DB. (s. f.-d). *OAuth2*. Part-DB Documentation. Recuperado el 24 de octubre de 2026, de https://docs.part-db.de/api/oauth.html

Part-DB. (s. f.-e). *Part-DB-server* [Software]. GitHub. Recuperado el 24 de octubre de 2026, de https://github.com/Part-DB/Part-DB-server

Part-DB. (2026, 27 de septiembre). *Version 2.19.0* [Software]. GitHub. https://github.com/Part-DB/Part-DB-server/releases/tag/v2.19.0

Zoho Corporation. (s. f.-a). *How do I get notified when the quantity of an item drops below the reorder level?* Zoho Inventory Knowledge Base. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/us/inventory/kb/items/item-reorder-notification.html

Zoho Corporation. (s. f.-b). *Introduction*. Zoho Inventory API Documentation. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/inventory/api/v1/introduction/

Zoho Corporation. (s. f.-c). *Inventory plans and pricing*. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/us/inventory/pricing/

Zoho Corporation. (s. f.-d). *OAuth*. Zoho Inventory API Documentation. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/inventory/api/v1/oauth/

Zoho Corporation. (s. f.-e). *Replenishments*. Zoho Inventory User Guide. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/us/inventory/help/items/replenishments.html

Zoho Corporation. (s. f.-f). *Reportes de inventario*. Ayuda de Zoho Inventory. Recuperado el 24

de octubre de 2026, de https://zoho.com/es-mx/inventory/help/reports/inventory-reports.html

Zoho Corporation. (s. f.-g). *Reports overview*. Zoho Inventory Help. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/us/inventory/help/reports/reports-overview.html

Zoho Corporation. (s. f.-h). *Software de gestión de inventario: Gestión de inventario en línea para empresas*. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/es-xl/inventory/

Zoho Corporation. (s. f.-i). *Users & roles*. Zoho Inventory Help. Recuperado el 24 de octubre de 2026, de https://www.zoho.com/au/inventory/help/settings/users.html

Zoho Corporation. (2026). *Zoho Inventory management app* (Versión 3.5.45) [Aplicación móvil]. App Store. https://apps.apple.com/app/id1037960494